Ferrolife

GPU-accelerated Game of Life in Rust

Charles Wang & Jason Liu
CIS 1905

Conway's Game of Life

Cellular automaton (James Conway, 1970) with rich space of emergent patterns

If cell is alive:

- Survives if # of live neighbors is 2 or 3
- Otherwise, dies from "underpopulation" or "overpopulation"

If cell is dead:

- Turns alive if # of live neighbors is 4 from "reproduction"
- Otherwise, remains dead

Extending Game of Life to RGB

Q: What does “alive” mean if cells are pixels in a colored image?

A: max(r, g, b) > alive_threshold

n.b. we experimented a lot with this question, and found this answer to be the most visually appealing and robust to user-uploaded image starting states

Q: What colors should “born” cells take?

A: Average of live neighbors’ colors

Q: What colors should “surviving” cells take?

A: Blend of its own and neighbors’ colors

wgpu: device creation

- Rust implementation of WebGPU, a graphics API for the web
  - Used in Firefox
  - Beyond the browser, can also be used and run natively on desktop
  - Calls underlying desktop graphics APIs (Vulkan, Metal, DirectX, etc.)
- The device lets you access most API functionality, but requires lots of initial setup
- Record/submit GPU commands using the device and queue

```rust
let instance = wgpu::Instance::new(wgpu::InstanceDescriptor {
    #[cfg(not(target_arch = "wasm32"))]
    backends: wgpu::Backends::PRIMARY,
    #[cfg(target_arch = "wasm32")]
    backends: wgpu::Backends::BROWSER_WEBGPU,
    flags: Default::default(),
    memory_budget_thresholds: Default::default(),
    backend_options: Default::default(),
    display: None,
});

let surface = instance.create_surface(window.clone()).unwrap();

let adapter = instance
    .request_adapter(&wgpu::RequestAdapterOptions {
        power_preference: wgpu::PowerPreference::HighPerformance,
        force_fallback_adapter: false,
        compatible_surface: Some(&surface),
    })
    .await?;

let (device, queue) = adapter
    .request_device(&wgpu::DeviceDescriptor {
        label: Some("device"),
        required_features: wgpu::Features::empty(),
        required_limits: wgpu::Limits::downlevel_defaults(),
        experimental_features: wgpu::ExperimentalFeatures::disabled(),
        memory_hints: Default::default(),
        trace: wgpu::Trace::Off,
    })
    .await?;

let window_size = window.inner_size();
```

wgpu: compute pass

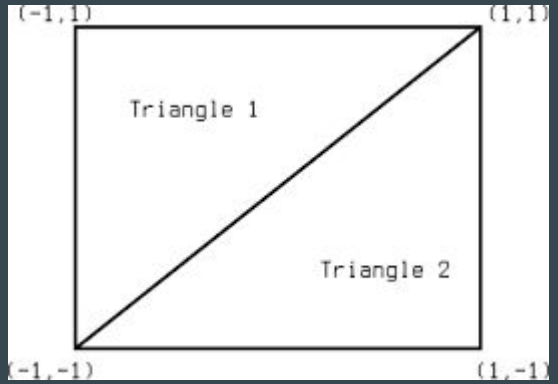
- Simulation steps are performed in parallel for each grid cell on the GPU
- Current grid state is stored in contiguous buffers in GPU memory
- Ping-pong buffers: can't read from and write to the same buffer, so every simulation step we read from one buffer and write to the other, then flip the direction for next time

```
1 implementation
pub struct Simulation {
    instances: Vec<Instance>,

    current_instance_buf: CurrentInstanceBuffer,
    state_buf_a: wgpu::Buffer,
    state_buf_b: wgpu::Buffer,

    ping_pong_bufs_bgl: wgpu::BindGroupLayout,
    grid_dims_bg: wgpu::BindGroup,
    pipeline: wgpu::ComputePipeline,
}
```

```
let (buf_written_to, buf_read_from) = match self.current_instance_buf {
    CurrentInstanceBuffer::A => {
        // Read from A, write to B
        self.current_instance_buf = CurrentInstanceBuffer::B;
        (&self.state_buf_b, &self.state_buf_a)
    }
    CurrentInstanceBuffer::B => {
        // Read from B, write to A
        self.current_instance_buf = CurrentInstanceBuffer::A;
        (&self.state_buf_a, &self.state_buf_b)
    }
};
```

wgpu: render pass

- Draw one mesh: a square made up of two triangles. That's a grid cell
- Use instanced drawing: share the same mesh but vary instance attributes, in this case model transformation matrix and color
- Consume the grid state buffer we modified in the compute pass




```
@vertex
fn vs_main(vert_in: VertexInput, inst_in: InstanceInput) -> VertexOutput {
    let instance_model = mat4x4f(
        inst_in.model_mat_0,
        inst_in.model_mat_1,
        inst_in.model_mat_2,
        inst_in.model_mat_3,
    );

    var out: VertexOutput;
    out.color = inst_in.color;
    out.clip_position = camera.view_proj * instance_model * vec4f(vert_in.position, 1.0);
    return out;
}

@fragment
fn fs_main(in: VertexOutput) -> @location(0) vec4f {
    return in.color;
}
```

Building for the web

- #[cfg] makes writing different platform code very easy! Much better than macros
- Program is compiled into WASM and draws to a <canvas> element
- Main event loop and async management is different

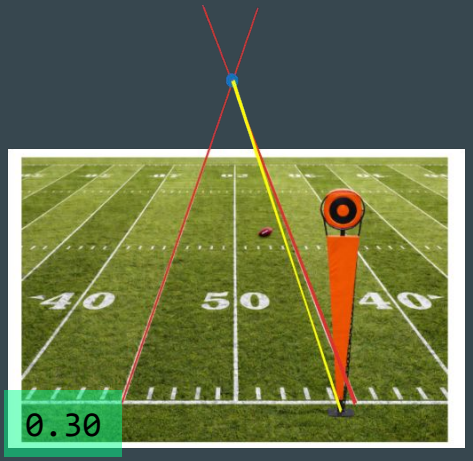
```rust
pub fn run() -> anyhow::Result<()> {
    #[cfg(not(target_arch = "wasm32"))]
    {
        env_logger::init();
    }
    #[cfg(target_arch = "wasm32")]
    {
        console_log::init_with_level(log::Level::Info).unwrap_throw();
    }

    let event_loop = EventLoop::<UserEvent>::with_user_event().build()?;

    #[cfg(not(target_arch = "wasm32"))]
    {
        let mut app = App::new(&event_loop);
        event_loop.run_app(&mut app)?;
    }
    #[cfg(target_arch = "wasm32")]
    {
        let app = App::new(&event_loop);
        event_loop.spawn_app(app);
    }

    Ok(())
}
```

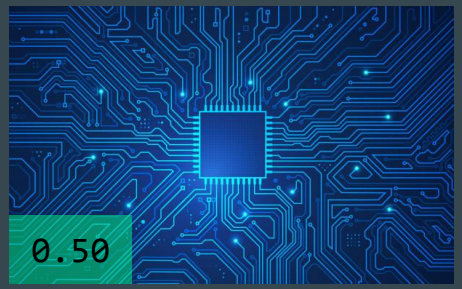
UI / QoL features

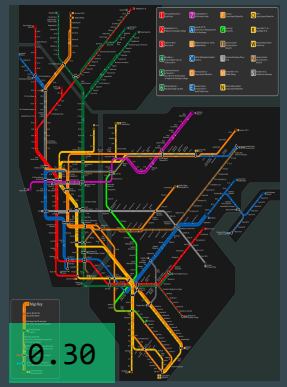
- Live interactive game board
  - Click to paint new cells
  - Slider to adjust liveness threshold
  - Upload / export board as image
- Simulation and camera controls
- Crates used:
  - **Rusty File Dialogs:** cross-platform open / save file dialogs
    - Requires running file dialog in separate thread
  - **ImGui:** Rust bindings for popular C++ GUI library
    - Utilizes builder struct pattern to account for optional C++ function parameters

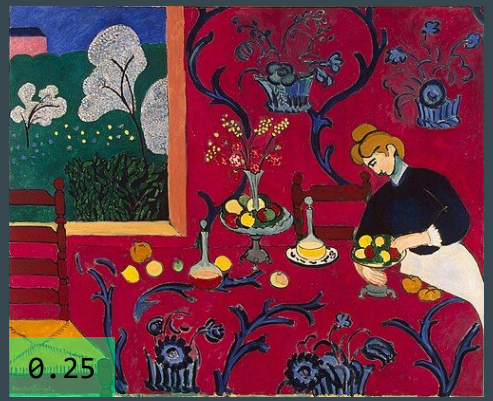
```
ui.window("Controls")
    .position([12.0, 12.0], Condition::FirstUseEver)
    .collapsed(true, Condition::FirstUseEver)
    .movable(false)
    .resizable(false)
    .always_auto_resize(true)
    .build(|| {
        let pause_label = if is_paused { "Resume" } el
        if ui.button(pause_label) {
            is_paused = !is_paused;
            reset_elapsed = true;
        }
        ui.same_line();
```

Simulation test images









Future extensions

- Infinite grid + HashLife
  - Challenge: GPU + quadtrees + hash data structure?
- Greater user control over simulation:
  - Adjustable simulation speed
  - Step between frames (forward and backward)
  - User-customizable game rules
- Dynamic game initialization:
  - Hard to determine best-looking threshold by hand
  - Given an initial image, automatically compute settings for visually interesting simulation

Final thoughts on Rust

- Coming from a C++ background, Rust has been really fun to use!
  - C++ if it had sensible defaults and chose to include standard library features in the core language
  - Build system and surrounding tooling has been really nice
  - Thinking about all the Rust features in terms of C++ has made understanding them easier
  - Module system lets us split Rust code in a more organized way, likely will scale better than C++ headers/source files in the long run
- You don't realize how often your program state is simply casing on a limited set of events, which the match expression is *very* useful for
- Dawn versus wgpu: have to deal with C++ warts, chose to implement WebGPU spec verbatim (more wordy!), does not include as many helpers as wgpu
- Everything being an expression (match, if/else, scope blocks, etc.) makes using and consuming values much easier and straightforward

Thank you!

Any questions?